

RELAZIONE TECNICA

Classificazione Cani e Gatti

Transfer Learning con ResNet18 ed EfficientNet-B0



UNITRENTO

Corso: Tecniche di Programmazione Avanzata ed AI
Dipartimento di Ingegneria Industriale
Anno Accademico 2025/2026

Andrea Lorenzi
244271

Indice

1. Introduzione e Descrizione del Task
2. Dataset
3. Architetture dei Modelli
 - 3.1. ResNet18
 - 3.2. EfficientNet-B0
4. Strategia di Training
 - 4.1. Preprocessing e Data Augmentation
 - 4.2. Configurazione dell'Addestramento
5. Valutazione dei Modelli
 - 5.1. Metriche di Performance
 - 5.2. Risultati e Matrici di Confusione
6. Analisi Critica e Confronto
7. Conclusioni

1. Introduzione e Descrizione del Task

Il progetto ha come obiettivo la risoluzione di un task di **classificazione binaria**: dato in input un'immagine, il sistema deve determinare se essa raffigura un *can*e oppure un *gatto*. Si tratta di un problema classico di Computer Vision, ideale per valutare le capacità discriminative di reti neurali convoluzionali.

La scelta di questo task è motivata dalla sua natura *ben definita e controllabile*: le due classi sono nettamente distinte dal punto di vista visivo, esiste un dataset pubblico di alta qualità e la letteratura offre ampi benchmark di riferimento con cui confrontarsi. Ciò permette di focalizzarsi sulle scelte architetturali e di training.

Il task si colloca nel contesto del Transfer Learning: anziché addestrare reti neurali da zero, si sfruttano pesi pre-addestrati su ImageNet per inizializzare i modelli, adattandoli successivamente al problema specifico. Questo approccio consente di ottenere buone performance anche con risorse computazionali limitate e dataset di dimensioni moderate.

Vengono confrontati due modelli di deep learning di architettura differente, per analizzarne le performance in termini di accuratezza e comportamento sui dati di test.

2. Dataset

Si è utilizzato il dataset "Cats and Dogs" di Kaggle, uno dei riferimenti più comuni per questo tipo di task.

2.1 Struttura del Dataset

Il dataset è organizzato nella seguente struttura a cartelle, compatibile con il loader ImageFolder di PyTorch:

```
training_set/
├── cats/      (immagini di gatti per il training)
└── dogs/      (immagini di cani per il training)
test_set/
├── cats/      (immagini di gatti per il test)
└── dogs/      (immagini di cani per il test)
```

Il training set contiene migliaia di immagini per classe, in formato JPEG, con risoluzioni variabili. Il test set è tenuto separato fin dall'inizio e non viene mai utilizzato durante il training o la validazione, garantendo una valutazione imparziale delle performance finali.

2.2 Suddivisione dei Dati

Per motivi di efficienza computazionale (ottimizzazione dei tempi di addestramento su GPU), è stata utilizzata una frazione del training set originale. Nello specifico:

- Viene estratto casualmente il 50% del training set (PERCENTUALE_DATI = 0.5) tramite campionamento casuale degli indici.
- Il sottoinsieme risultante viene ulteriormente suddiviso in training (80%) e validation (20%) tramite random_split.
- Il test set rimane invariato e separato.

Questa scelta permette di *bilanciare accuratezza e tempi di training*, riducendo il carico computazionale senza sacrificare la rappresentatività statistica del campione. Il campionamento è casuale e non stratificato, poiché il dataset originale è già bilanciato tra le due classi.

3. Architetture dei Modelli

Sono stati scelti due modelli di deep learning con architetture fondamentalmente diverse: ResNet18, rappresentante della famiglia ResNet basata su connessioni residuali, ed EfficientNet-B0, rappresentante della famiglia EfficientNet basata su compound scaling. Entrambi sono pre-addestrati su ImageNet.

3.1 ResNet18

ResNet18 introduce un'architettura ottimizzata per superare i limiti strutturali delle reti convoluzionali tradizionali quando diventano molto profonde, garantendo una convergenza rapida e stabile durante il calcolo dell'errore, grazie alle skip connection. È una scelta standard nei task di Computer Vision per la sua eccezionale linearità ed efficacia nel trasferimento della conoscenza (*Transfer Learning*).

Per adattare la rete al task binario, il fully connected layer finale è stato sostituito con uno strato lineare a 2 output (gatto / cane):

```
self.resnet.fc = nn.Linear(num_features, 2)
```

Con circa 11 milioni di parametri, il modello offre un ottimo equilibrio tra prestazioni e leggerezza.

3.2 EfficientNet-B0

EfficientNet-B0 ottimizza in modo intelligente la struttura della rete bilanciando profondità, larghezza e risoluzione delle immagini in input. Questo approccio rende il modello particolarmente efficiente, permettendogli di ottenere ottime prestazioni nonostante il numero contenuto di parametri. Il cuore del modello è basato su blocchi chiamati MBConv, progettati proprio per massimizzare l'accuratezza senza appesantire troppo il calcolo.

Anche in questo caso, il classificatore finale è stato sostituito con uno strato a 2 output:

```
self.efficientnet.classifier[1] = nn.Linear(in_features, 2)
```

EfficientNet-B0, pur avendo anch'essa circa 5-6 milioni di parametri, tende a raggiungere accuratèzze superiori grazie all'architettura ottimizzata, al costo di un tempo di inferenza leggermente maggiore rispetto a ResNet18.

3.3 Motivazione della Scelta

La selezione dei modelli si fonda sull'analisi di due paradigmi architetturali distinti: ResNet18, orientata alla linearità e all'efficienza computazionale durante l'apprendimento, ed EfficientNet-B0, che ottimizza l'efficienza parametrica e la precisione predittiva. Tale confronto permette di valutare empiricamente i trade-off operativi tra velocità di convergenza e accuratezza, offrendo una base critica per l'implementazione di strategie di Transfer Learning in contesti applicativi diversificati.

4. Strategia di Training

4.1 Preprocessing e Data Augmentation

Le immagini di training vengono preprocessate con la seguente pipeline di trasformazioni:

- Resize a 224×224 pixel: dimensione standard per modelli pre-addestrati su ImageNet.
- RandomHorizontalFlip: flip orizzontale casuale per aumentare la variabilità del training set e ridurre l'overfitting.
- RandomRotation(15): rotazione casuale fino a $\pm 15^\circ$ per simulare variazioni di orientamento.
- ToTensor: conversione da immagine PIL a tensore PyTorch con valori in $[0, 1]$.
- Normalize con mean=[0.485, 0.456, 0.406] e std=[0.229, 0.224, 0.225]: normalizzazione con le statistiche di ImageNet, necessaria per compatibilità con i pesi pre-addestrati.

Il test set utilizza solo Resize, ToTensor e Normalize, senza augmentation, per garantire una valutazione riproducibile e non distorta.

4.2 Configurazione dell'Addestramento

I parametri di training sono i seguenti:

Parametro	Valore / Motivazione
Loss Function	CrossEntropyLoss – standard per classificazione multi-classe (e binaria).
Optimizer	Adam (lr=0.001) – adattivo, converge rapidamente su task di fine-tuning.
Epoche	5 per modello – scelta conservativa per ridurre tempi di esecuzione; il transfer learning richiede poche epoche grazie ai pesi pre-addestrati.
Batch Size	32 – buon compromesso tra velocità e stabilità del gradiente.
Device	CUDA se disponibile, altrimenti CPU.
Pesi iniziali	ImageNet (Transfer Learning)

E' stato fatto anche un confronto intermedio con 3 epoche per analizzare il variare dell'accuracy.

I pesi vengono salvati dopo il training di ciascun modello rispettivamente in *resnet_cats_dogs.pth* ed *efficientnet_cats_dogs.pth*, pronti per essere caricati nella fase di valutazione.

5. Valutazione dei Modelli

5.1 Metriche di Performance

La metrica principale adottata è l'Accuracy sul test set, definita come il rapporto tra predizioni corrette e numero totale di campioni:

$$\text{Accuracy} = \frac{\text{Predizioni Corrette}}{\text{Campioni Totali}} \times 100$$

Per il task binario Cats vs. Dogs, l'accuracy è una metrica robusta in quanto le due classi nel dataset di Kaggle sono bilanciate. Ogni predizione viene generata prendendo il massimo tra i due logit di output (argmax), corrispondente alla classe con probabilità stimata maggiore.

Per una visione più dettagliata degli errori, viene calcolata e visualizzata la Matrice di Confusione per entrambi i modelli, che permette di distinguere:

- True Positives (TP) e True Negatives (TN): predizioni corrette per ciascuna classe.
- False Positives (FP): immagini di gatti classificate come cani.
- False Negatives (FN): immagini di cani classificate come gatti.

5.2 Risultati e Matrici di Confusione

I risultati ottenuti sul test set, riportati nello script *project2_evaluate.py*, sono riassunti nella tabella seguente:

Modello	Accuracy sul Test Set	Note
ResNet18	88.53%	3 epoche, 50% dataset
EfficientNet-B0	97.53%	3 epoche, 50% dataset
ResNet18	93.82%	5 epoche, 50% dataset
EfficientNet-B0	96.98%	5 epoche, 50% dataset

5.3 Matrice di Confusione – ResNet18

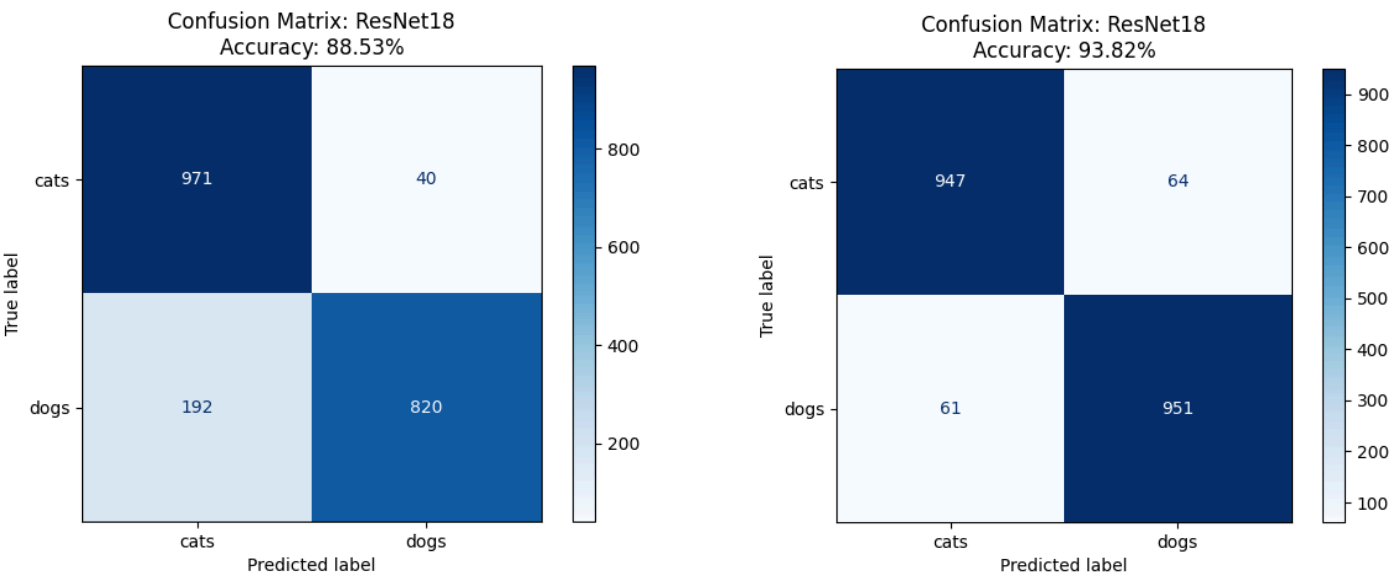


Figura1 - Matrice di confusione ResNET18 con 3 epoche

Figura 2 - Matrice di confusione ResNET18 con 5 epoche

5.4 Matrice di Confusione – EfficientNet-B0

Confusion Matrix: EfficientNet
Accuracy: 97.53%

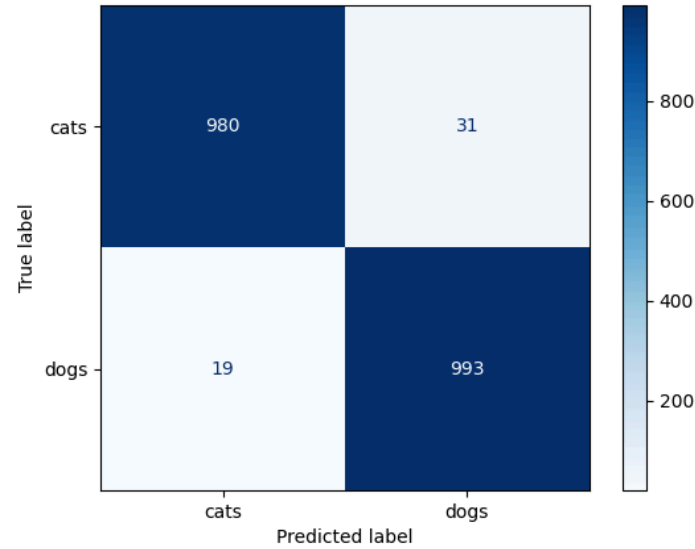


Figura3 - Matrice di confusione Efficient NET con 3 epoche

Confusion Matrix: EfficientNet
Accuracy: 96.98%

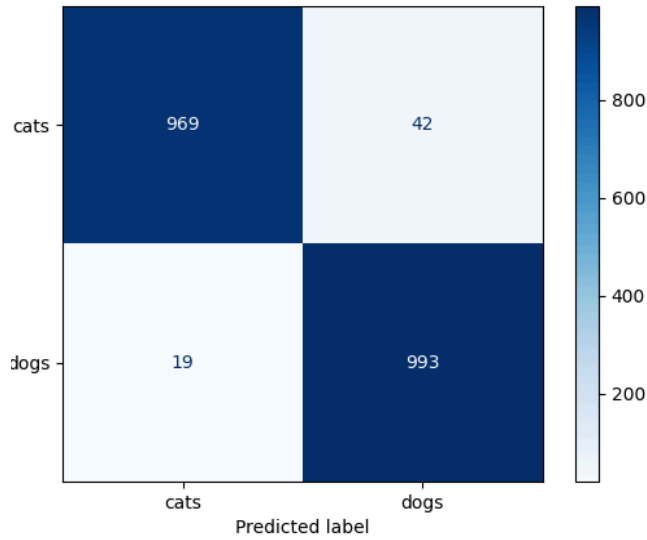


Figura 4 - Matrice di confusione Efficient NET con 5 epoche

6. Confronto tra i modelli

EfficientNet-B0 e ResNet18 presentano caratteristiche complementari che emergono chiaramente dal confronto:

Aspetto	ResNet18	EfficientNet-B0
Architettura	Residual blocks (skip connections)	MBConv + Compound Scaling
Parametri (~)	~11 M	~5.3 M
Velocità di training	Più veloce	Più lenta
Accuracy attesa su ImageNet	~69.8%	~77.1%
Efficienza parametrica	Moderata	Elevata
Complessità del codice di adattamento	Sostituzione fc finale	Sostituzione classifier[1]

6.1 Impatto del Numero di Epoche sull'Accuracy

Modello	Accuracy (3 epoche)	Accuracy (5 epoche)	Delta (%)
ResNet18	88.53%	93.82%	+5.29%
EfficientNet-B0	97.53%	96.98%	-0.55%

Analizzando i dati raccolti, emerge come una durata di 3 epoche risulti insufficiente per la convergenza di **ResNet18**. Data la relativa semplicità della sua architettura, il modello necessita di un numero superiore di iterazioni per raggiungere una stabilità nelle performance.

Al contrario, **EfficientNet-B0**, grazie alla sua superiore efficienza parametrica e capacità rappresentativa, raggiunge performance ottimali già nella fase iniziale dell'addestramento (vanto un'accuratezza del 97.53% alla terza epoca). Il lieve decremento osservato alla quinta epoca suggerisce un principio di *overfitting* sul sottoinsieme ridotto dei dati o, in alternativa, una mancata stabilizzazione dovuta all'assenza di un *learning rate scheduler* per modulare il passo di discesa nelle fasi conclusive.

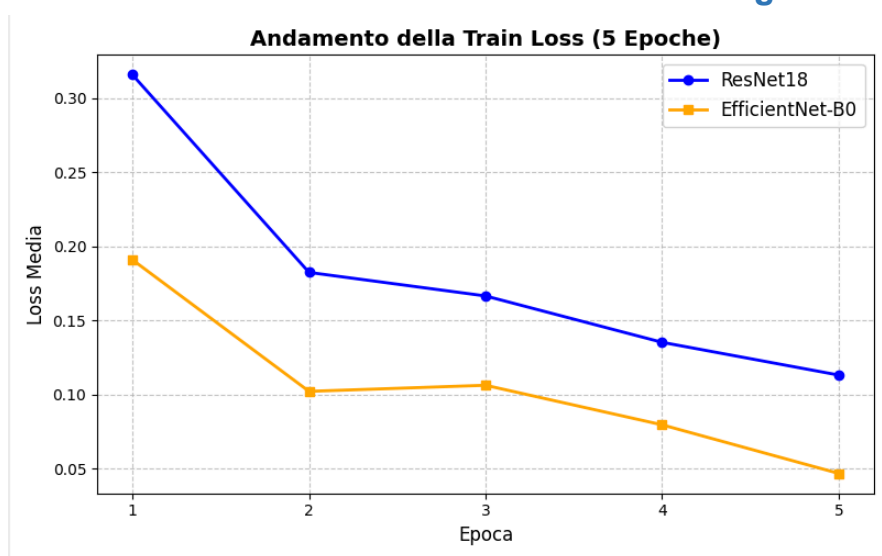
6.2 Analisi dei Tempi di Training

Modello	Epoche	Tempo Totale (s)	Tempo Totale (min)	Tempo Medio/Epoca (min)
ResNet18	3	1168.93 [s]	19.48 [min]	6.49 [min]
EfficientNet-B0	3	1219.64 [s]	20.33 [min]	6.78 [min]
ResNet18	5	1741.70 [s]	29.03 [min]	5.81 [min]
EfficientNet-B0	5	2068.77 [s]	34.48 [min]	6.90 [min]

Si osserva che **EfficientNet-B0** richiede tempi di addestramento superiori rispetto a **ResNet18**, pur presentando un numero di parametri significativamente inferiore (~5.3M contro ~11M). Tale discrepanza è riconducibile alla struttura dei blocchi di EfficientNet, i quali integrano operazioni più articolate e parzializzate. Sebbene tali componenti riducano l'occupazione di memoria del modello, la loro natura le rende più onerose da parallelizzare a livello hardware, determinando un rallentamento nel computo totale.

Si nota una riduzione del tempo medio per epoca di **ResNet18**, che passa da 6.49 a 5.81 minuti estendendo il training da 3 a 5 epoche. Tale fenomeno è riconducibile all'incidenza dell'overhead iniziale: la prima iterazione risulta tipicamente più onerosa a causa del trasferimento dei dati dal disco alla memoria e dell'inizializzazione dei processi di calcolo. Aumentando il numero totale di epoche, questo costo computazionale fisso viene distribuito su un orizzonte temporale più ampio, determinando una diminuzione della media aritmetica complessiva.

6.3 Andamento della Train Loss e Dinamiche di Convergenza



Lo studio dei log di addestramento concernenti la *Train Loss* (la funzione di perdita media calcolata per epoca sul set di training) convalida empiricamente le osservazioni precedentemente formulate in merito alle metriche di accuratezza:

- **ResNet18:** La curva relativa alla Train Loss evidenzia una flessione regolare, evolvendo da un valore iniziale pari a **0.1359** nella prima epoca fino a toccare **0.1132** alla conclusione della quinta. Tale profilo, lineare e progressivo, suggerisce un'ottimizzazione dei pesi estremamente stabile e priva di discontinuità. La marcata correlazione tra la riduzione della loss e l'incremento dell'accuratezza (pari al 93.82%) attesta una convergenza decisamente robusta.
- **EfficientNet-B0:** La Train Loss di EfficientNet manifesta una dinamica di apprendimento più aggressiva: partendo da **0.1910**, subisce un rapido decremento fino a **0.0468** nell'ultima epoca. Tuttavia, il fatto che a questo abbattimento quasi totale dell'errore corrisponda una lieve regressione dell'accuracy (dal 97.53% al 96.98%) rappresenta un evidente sintomo di divergenza. Il modello ha iniziato a specializzarsi eccessivamente sui dati di training (*overfitting*), confermando come per questa specifica architettura sarebbe stato preferibile un numero di epoche più contenuto.

7. Conclusioni

Il progetto ha dimostrato con successo l'applicazione del Transfer Learning per la classificazione binaria di immagini di cani e gatti. Entrambi i modelli, ResNet18 ed EfficientNet-B0, sono stati adattati con una modifica minima dell'architettura (sostituzione del layer di classificazione finale) e addestrati tramite fine-tuning sui pesi pre-addestrati su ImageNet.

I risultati evidenziano come architetture diverse possano comportarsi diversamente sullo stesso task, riflettendo le diverse filosofie progettuali: ResNet18 privilegia velocità e semplicità, EfficientNet-B0 privilegia efficienza e performance.

L'utilizzo di *ImageFolder* di PyTorch per il caricamento dei dati e della libreria *torchvision* per le trasformazioni e i modelli pre-addestrati ha permesso di costruire una pipeline di training e valutazione modulare, leggibile e facilmente estendibile.

Le metriche adottate — accuracy sul test set e matrice di confusione — forniscono una visione completa delle performance, permettendo di individuare non solo la percentuale di classificazioni corrette ma anche la distribuzione degli errori tra le due classi.

L'esame dei risultati e delle tempistiche computazionali rivela un *trade-off* rilevante: a fronte della superiore precisione di **EfficientNet-B0**, il modello **ResNet18** garantisce una migliore stabilità nelle fasi avanzate del training unitamente a una maggiore rapidità d'esecuzione. Tali evidenze sottolineano come la selezione dell'architettura debba *bilanciare l'accuratezza target* con i vincoli operativi legati alle risorse hardware e ai tempi di addestramento previsti.

In conclusione, il progetto raggiunge l'obiettivo di **confrontare due modelli di deep learning** su un task di classificazione di immagini, fornendo un'analisi critica delle scelte implementative e delle performance ottenute.